The AT Protocol as Composable Session Types

A Case Study in Protocol-as-Type Composition

The Prologos Project - from Zachary Larson

2026-03-05

Contents

1 Abstract **3**

2 1. AT Protocol Architecture Overview **3**
2.1 1.1 Key Participants 4
2.2 1.2 Key Data Types 4

3 2. Identity Resolution as Session Types **5**
3.1 2.1 Handle Resolution Phase 5
3.2 2.2 DID Document Resolution Phase 6
3.3 2.3 Bidirectional Verification (Composed) 6
3.4 2.4 Analysis 7

4 3. OAuth Authentication as Session Types **7**
4.1 3.1 Foundational Types 7
4.2 3.2 Server Discovery Phase 8
4.3 3.3 Pushed Authorization Request Phase 8
4.4 3.4 Token Exchange Phase 9
4.5 3.5 Token Refresh Phase 9
4.6 3.6 Full OAuth Flow (Composed) 9
4.7 3.7 Analysis 10

5 4. XRPC Request/Response as Session Types **11**
5.1 4.1 XRPC Types 11
5.2 4.2 Query Session (Cacheable, No Side Effects) 11
5.3 4.3 Procedure Session (May Mutate State) 12
5.4 4.4 Paginated Query (Recursive) 12
5.5 4.5 Analysis 12

6 5. Repository Operations as Session Types **13**
6.1 5.1 Repository Types 13
6.2 5.2 Record CRUD Session 13
6.3 5.3 Analysis: Capabilities 14

7 6. Data Sync and Firehose as Session Types **15**
7.1 6.1 Sync Event Types 15
7.2 6.2 Firehose Subscription Session 16
7.3 6.3 Full Repository Sync Session 17
7.4 6.4 Relay Aggregation (Multi-Source Composition) 17
7.5 6.5 Analysis: Protocol Reuse 18

8 7. Account Lifecycle as Session Types **18**
8.1 7.1 Account State Machine 19
8.2 7.2 Account Migration (Cross-PDS) 19
8.3 7.3 Analysis: Capability Tracking Through Migration 21

9 8. Blob Management as Session Types **21**

10 9. Lexicon Validation as a Type-Level Concern **22**
10.1 9.1 Lexicon Schema as Type (Sketch) 23

11 10. The Full Client Session: Composition at Scale **24**
11.1 10.1 Client PDS Full Session 24
11.2 10.2 The Protocol Graph 26

12 11. Federation as Multi-Party Protocol Composition **27**
12.1 11.1 Analysis: Same Protocol, Different Roles 28

13 12. Key Findings **28**
13.1 12.1 Every ATP Sub-Protocol Maps to a Session Type 28
13.2 12.2 Composition Patterns Observed 29
13.3 12.3 What Session Types Catch That ATP Cannot 29
13.4 12.4 Dependent Types Add Schema Safety 30
13.5 12.5 Capability Integration Is Natural 30

14 13. Implications for Prologos **31**
14.1 13.1 Protocols-as-Types Is Practical 31
14.2 13.2 Session Type Libraries Enable Protocol Ecosystems . . . 31
14.3 13.3 Federation Is Natural 31
14.4 13.4 Dependent Sessions Enable Schema-Safe APIs 31

14.5 13.5 The Intentionality Principle Holds 32

15 14. References **32**

15.1 Prologos References 32

1 Abstract

The Authenticated Transfer Protocol (AT Protocol, atproto) is a decentralized social networking protocol powering Bluesky and its federation network. It comprises multiple interlocking sub-protocols: identity resolution, authentication, repository synchronization, XRPC request/response, and real-time event streaming.

This document demonstrates that the entire AT Protocol can be expressed as composable session types in Prologos, using the protocols-as-types composition mechanism described in `PROTOCOLS_AS_TYPES.org`. Each sub-protocol becomes a named session type; complex interactions are assembled from these building blocks through named continuations. Capabilities track authority through every phase transition.

The result is a complete, type-checked specification of a real-world distributed protocol — one that would catch protocol violations, state machine errors, and capability misuse at compile time.

2 1. AT Protocol Architecture Overview

The AT Protocol is a layered architecture for decentralized social networking:

```
                Applications
        (Bluesky, feeds, labelers)

            Lexicon (Schema Layer)
   NSID-keyed schemas, record types, RPCs

        XRPC (HTTP API Layer)
   Query (GET) / Procedure (POST) / Stream

       Identity (DID + Handle Layer)
  did:plc / did:web, DNS handles, key rotation
```

```
   Repository (Data Layer)
MST, signed commits, CAR exports, records

     Sync (Federation Layer)
Event streams (firehose), repo diffs, relays

 Auth (OAuth 2.1 + DPoP + PKCE)
PAR, token exchange, scope management
```

2.1 1.1 Key Participants

Role	Description	ATP Name
User Client	Browser or mobile app	Client
Personal Server	Hosts user's repo, auth, blobs	PDS
Relay	Aggregates firehoses into network stream	Relay (BGS)
App View	Application-specific indexing/aggregation	App View
Labeler	Content moderation metadata service	Labeler

2.2 1.2 Key Data Types

```
;; --- AT Protocol Core Data Types ---

type DID := String                      ;; did:plc:ewvi7nxzyoun6zhxrhs64oiz
type Handle := String                   ;; user.bsky.social
type NSID := String                     ;; com.atproto.repo.createRecord
type TID := String                      ;; timestamp-based ID (logical clock)
type CID := String                      ;; content-addressed identifier
type ATUri := String                    ;; at://did:plc:xxx/app.bsky.feed.post/tid

type AtIdentifier := DID | Handle

type SigningKey
  :public-key Bytes
  :algorithm  KeyAlgorithm

type KeyAlgorithm := :p256 | :k256

type DIDDocument
  :did              DID
```

```
  :also-known-as    [List ATUri]         ;; handle claims
  :verification     [List VerificationMethod]
  :service          [List ServiceEndpoint]

type VerificationMethod
  :id               String               ;; e.g. did:plc:xxx#atproto
  :type             String               ;; "Multikey"
  :public-key       String               ;; multibase-encoded

type ServiceEndpoint
  :id               String               ;; e.g. did:plc:xxx#atproto_pds
  :type             String               ;; "AtprotoPersonalDataServer"
  :endpoint         String               ;; https://pds.example.com
```

3 2. Identity Resolution as Session Types

Identity resolution in atproto is a multi-step verification process: handle → DID → DID document → verify handle claim (bidirectional).

This is our first composition target. The resolution process has distinct phases, each of which becomes a named session type.

3.1 2.1 Handle Resolution Phase

```
;; Client resolves a handle to a DID via DNS TXT or HTTPS well-known.
;; The resolver is a service that performs the lookup.

session HandleResolve
  ! Handle                             ;; client sends the handle to resolve
  ? [Result DID ResolveError]          ;; service returns DID or error
  end

type ResolveError
  := :dns-failure
   | :no-txt-record
   | :https-failure
   | :invalid-did
   | :handle-not-found
```

3.2 2.2 DID Document Resolution Phase

```
;; Given a DID, resolve to a DID Document.
;; For did:plc → PLC directory lookup.
;; For did:web → HTTPS .well-known/did.json fetch.

session DIDResolve
  ! DID                                    ;; client sends DID
  ? [Result DIDDocument ResolveError]      ;; service returns document or error
  end
```

3.3 2.3 Bidirectional Verification (Composed)

The full identity verification composes both phases and adds a verification step. This is where protocol composition shines:

```
;; Full identity verification: resolve handle, resolve DID, verify bidirectional claim
;; This is NOT end-spliced - it's an inline prefix with a named continuation.

session IdentityVerification
  ;; Phase 1: Handle → DID
  ! Handle
  ? [Result DID ResolveError] .
  +>
    | :resolved ->
        ;; Phase 2: DID → Document
        ! DID
        ? [Result DIDDocument ResolveError] .
        +>
          | :doc-resolved ->
              ;; Phase 3: Verify handle  document.also-known-as
              ? [Result VerifiedIdentity VerificationError]
              end
          | :doc-failed -> end
    | :handle-failed -> end

type VerifiedIdentity
  :did        DID
  :handle     Handle
  :pds-url    String
  :signing-key SigningKey
```

```
type VerificationError
  := :handle-not-in-document            ;; DID doc doesn't claim this handle
   | :signing-key-missing               ;; no #atproto verification method
   | :pds-endpoint-missing              ;; no #atproto_pds service
```

3.4 2.4 Analysis

The `IdentityVerification` session type encodes the AT Protocol's critical security property — bidirectional verification — directly in the type. A process implementing this protocol **cannot** skip the verification step. The type checker guarantees that if the identity is returned as `VerifiedIdentity`, both the handle→DID and DID→handle directions have been checked.

Note the branching structure: each phase can fail independently. The session type's choice operators (`+>`) make the failure modes explicit in the type. A consumer of this protocol sees exactly which phases can fail and must handle each case.

4 3. OAuth Authentication as Session Types

The AT Protocol's OAuth 2.1 flow is a complex multi-phase protocol involving client discovery, PAR (Pushed Authorization Requests), PKCE challenge, user consent, and token exchange with DPoP binding.

4.1 3.1 Foundational Types

```
type ClientId := String                 ;; https://app.example.com/oauth-client-metada
type AuthCode := String
type AccessToken := String
type RefreshToken := String
type DPoPNonce := String
type PKCEChallenge := String
type PKCEVerifier := String
type Scope := String                    ;; "atproto", "transition:generic", etc.
type RequestUri := String               ;; PAR reference

type TokenResponse
  :access-token   AccessToken
  :refresh-token  RefreshToken
  :token-type     String                ;; "DPoP"
```

```
  :expires-in     Int
  :scope          String

type AuthError
  := :invalid-client
   | :invalid-grant
   | :invalid-scope
   | :access-denied
   | :server-error
   | :temporarily-unavailable
```

4.2 3.2 Server Discovery Phase

```
;; Discover the Authorization Server for a given PDS.
;; Client resolves /.well-known/oauth-authorization-server

session AuthServerDiscovery
  ! String                                ;; PDS URL
  ? [Result AuthServerMetadata AuthError]
  end

type AuthServerMetadata
  :issuer                 String
  :authorization-endpoint String
  :token-endpoint         String
  :par-endpoint           String
  :dpop-signing-alg       [List String]  ;; ["ES256"]
  :scopes-supported       [List Scope]
```

4.3 3.3 Pushed Authorization Request Phase

```
;; PAR: client submits authorization parameters server-side,
;; receives a request_uri for the redirect.

session PushedAuthRequest
  ! ClientId
  ! [List Scope]                          ;; requested scopes
  ! PKCEChallenge                         ;; S256 challenge
  ! String                                ;; redirect_uri
  ? [Result RequestUri AuthError]
```

```
  end
```

4.4 3.4 Token Exchange Phase

```
;; After user consent, client exchanges auth code for tokens.
;; DPoP proof binds tokens to this client instance.

session TokenExchange
  ! AuthCode                              ;; authorization code from redirect
  ! PKCEVerifier                          ;; proof of PKCE challenge
  ! Bytes                                 ;; DPoP proof JWT
  ? [Result TokenResponse AuthError]
  end
```

4.5 3.5 Token Refresh Phase

```
;; Refresh an expired access token using the refresh token.
;; DPoP nonce may rotate.

session TokenRefresh
  rec
    ! RefreshToken
    ! Bytes                               ;; DPoP proof
    ? [Result TokenResponse AuthError] .
    +>
      | :refreshed -> TokenRefresh        ;; can refresh again later
      | :expired   -> end                 ;; refresh token expired, re-auth needed
```

4.6 3.6 Full OAuth Flow (Composed)

Now we compose the phases into the full OAuth flow. This is the key demonstration: each phase is a named session type, and the full flow sequences them through named continuations.

```
;; The full OAuth flow composes four phases.
;; Note: user consent happens out-of-band (browser redirect),
;; so the session models the clientserver wire protocol, not the UI.

session OAuthFlow
  ;; Phase 1: Discover auth server
  ! String                                ;; PDS URL
```

```
  ? [Result AuthServerMetadata AuthError] .
  +>
    | :discovered ->
        ;; Phase 2: PAR
        ! ClientId . ! [List Scope] . ! PKCEChallenge . ! String .
        ? [Result RequestUri AuthError] .
        +>
          | :par-ok ->
              ;; [User consent happens out-of-band via browser redirect]
              ;; Phase 3: Token exchange
              ! AuthCode . ! PKCEVerifier . ! Bytes .
              ? [Result TokenResponse AuthError] .
              +>
                | :tokens-granted -> AuthenticatedSession
                | :exchange-failed -> end
          | :par-failed -> end
    | :discovery-failed -> end

;; After successful OAuth, transition to authenticated operations
session AuthenticatedSession
  +>
    | :xrpc-query    -> XRPCQuery . AuthenticatedSession
    | :xrpc-procedure -> XRPCProcedure . AuthenticatedSession
    | :refresh-token -> TokenRefresh
    | :logout        -> end
```

4.7 3.7 Analysis

The composed `OAuthFlow` session type captures the **entire** OAuth 2.1 + DPoP + PKCE flow as a single verifiable type. Key observations:

1. **Phase sequencing is enforced**: you cannot exchange tokens before completing PAR, and you cannot do PAR before discovering the auth server. The type prevents out-of-order operations.

2. **Error handling is explicit at every phase**: each phase can fail independently. The session type's branching makes it impossible to skip error handling.

3. **The transition to `AuthenticatedSession`** is a named continuation. After successful authentication, the protocol transitions to a new mode

where XRPC queries and procedures are available. This mode transition is type-checked.

4. **Token refresh is recursive**: `TokenRefresh` recurses back to itself, modeling the repeated refresh cycle. When the refresh token expires, the session terminates — the client must re-authenticate.

5 4. XRPC Request/Response as Session Types

XRPC is the HTTP API layer. Queries (GET) and Procedures (POST) are the two operation types.

5.1 4.1 XRPC Types

```
type XRPCMethod := NSID                  ;; e.g. com.atproto.repo.createRecord

type XRPCParams := Map String String     ;; query parameters

type XRPCError
  :error   String                        ;; error type name
  :message String?                       ;; optional human-readable message

type HTTPStatus := Int

type XRPCResponse (A : Type)
  := :ok A
   | :client-error HTTPStatus XRPCError  ;; 400, 401, 403, 404, 413
   | :rate-limited Int?                  ;; 429 + optional Retry-After seconds
   | :server-error HTTPStatus XRPCError  ;; 500, 501, 502, 503, 504
```

5.2 4.2 Query Session (Cacheable, No Side Effects)

```
;; An XRPC query: client sends method + params, server responds.
;; Queries are idempotent and cacheable (HTTP GET semantics).

session XRPCQuery
  ! XRPCMethod                           ;; method NSID
  ! XRPCParams                           ;; query parameters
  ? [XRPCResponse Bytes]                 ;; response (JSON or CBOR body)
  end
```

5.3 4.3 Procedure Session (May Mutate State)

```
;; An XRPC procedure: client sends method + params + body,
;; server may mutate state and responds.

session XRPCProcedure
  ! XRPCMethod                          ;; method NSID
  ! XRPCParams                          ;; query parameters
  ! Bytes                               ;; request body
  ? [XRPCResponse Bytes]                ;; response body
  end
```

5.4 4.4 Paginated Query (Recursive)

```
;; Many XRPC queries support cursor-based pagination.
;; This session type models the pagination loop.

type CursorResponse (A : Type)
  :data   A
  :cursor String?                       ;; none = no more pages

session PaginatedQuery
  ! XRPCMethod . ! XRPCParams .
  ? [XRPCResponse [CursorResponse Bytes]] .
  +>
    | :next-page ->
        ! String .                      ;; cursor for next page
        ? [XRPCResponse [CursorResponse Bytes]] .
        PaginatedQuery                  ;; recurse: may have more pages
    | :done -> end
```

5.5 4.5 Analysis

Note how `PaginatedQuery` uses self-reference for recursion: the client can keep requesting pages until it selects `:done`. The session type guarantees that pagination follows the cursor protocol — you can't request page 3 without having received the cursor from page 2.

The `XRPCResponse` union type captures all possible HTTP outcomes, forcing the consumer to handle rate limiting (429), authentication failures (401), and server errors.

6 5. Repository Operations as Session Types

The AT Protocol repository is a Merkle Search Tree (MST) of records, signed with the account's key. Repository operations include creating, updating, and deleting records.

6.1 5.1 Repository Types

```
type RepoCommit
  :did     DID
  :version Int                          ;; always 3
  :data    CID                          ;; MST root
  :rev     TID                          ;; revision (logical clock)
  :prev    CID?                         ;; previous commit (typically none in v3)
  :sig     Bytes                        ;; cryptographic signature

type RecordKey := String                ;; typically a TID
type Collection := NSID                 ;; e.g. app.bsky.feed.post

type RepoPath
  :collection Collection
  :rkey       RecordKey

type RepoRecord
  :uri     ATUri
  :cid     CID
  :value   Bytes                        ;; CBOR-encoded record

type RepoOp
  := :create RepoPath Bytes             ;; path + record value
   | :update RepoPath Bytes CID         ;; path + new value + expected CID (swap)
   | :delete RepoPath
```

6.2 5.2 Record CRUD Session

```
;; Client  PDS session for repository record operations.
;; This models the com.atproto.repo.* family of XRPC endpoints
;; as a single session type with choice.

session RepoSession {auth :0 AuthCap}
```

```
  rec
    +>
      | :create-record ->
          ! Collection . ! RecordKey? . ! Bytes .
          ? [Result RepoRecord RepoError] . RepoSession
      | :get-record ->
          ! Collection . ! RecordKey .
          ? [Result RepoRecord RepoError] . RepoSession
      | :update-record ->
          ! Collection . ! RecordKey . ! Bytes . ! CID .
          ? [Result RepoRecord RepoError] . RepoSession
      | :delete-record ->
          ! Collection . ! RecordKey .
          ? [Result Unit RepoError] . RepoSession
      | :list-records ->
          ! Collection . ! Int . ! String? .     ;; collection, limit, cursor
          ? [Result [CursorResponse [List RepoRecord]] RepoError] . RepoSession
      | :describe ->
          ? [Result RepoDescription RepoError] . RepoSession
      | :done -> end

type RepoError
  := :invalid-record                    ;; record fails Lexicon validation
   | :record-not-found
   | :conflict CID                      ;; swap CID mismatch (optimistic concurrency)
   | :repo-not-found
   | :unauthorized
   | :rate-limited

type RepoDescription
  :handle           Handle
  :did              DID
  :collections      [List NSID]
  :did-doc          DIDDocument
```

6.3 5.3 Analysis: Capabilities

Note the `{auth :0 AuthCap}` on `RepoSession`. This is a capability constraint: the session requires proof of authentication. It is `:0` (erased) because it's an authority check, not a runtime value. The type checker verifies that

any process operating this session has been authenticated — but there is zero runtime cost.

Write operations (`:create-record`, `:update-record`, `:delete-record`) could further require `{write :0 RepoWriteCap}`, enabling fine-grained authority separation between read and write operations.

7 6. Data Sync and Firehose as Session Types

The sync layer is the heart of AT Protocol federation. PDS instances emit repository events; Relays aggregate them; App Views consume them.

7.1 6.1 Sync Event Types

```
type SyncEvent
  := :commit CommitEvent
   | :identity IdentityEvent
   | :account AccountEvent
   | :sync SyncResetEvent

type CommitEvent
  :repo    DID
  :rev     TID
  :since   TID?                          ;; previous rev (for chain verification)
  :blocks  Bytes                         ;; CAR diff slice ( 2 MB)
  :ops     [List RecordOp]              ;; record-level operations ( 200)
  :commit  CID
  :time    String                        ;; ISO 8601

type RecordOp
  :action    RecordAction
  :path      String                      ;; collection/rkey
  :cid       CID?                        ;; record CID (none for deletes)

type RecordAction := :create | :update | :delete

type IdentityEvent
  :did     DID
  :seq     Int
  :handle  Handle?                       ;; current handle (may have changed)
  :time    String
```

```
type AccountEvent
  :did     DID
  :seq     Int
  :active  Bool
  :status  AccountStatus?
  :time    String

type AccountStatus
  := :takendown | :suspended | :deleted | :deactivated

type SyncResetEvent
  :did     DID
  :rev     TID
  :commit  Bytes                        ;; signed commit block (no records)
  :time    String
```

7.2 6.2 Firehose Subscription Session

This is the most interesting session type in the entire AT Protocol from a composition standpoint. The firehose is a long-lived, server-push stream with cursor-based resumption.

```
;; com.atproto.sync.subscribeRepos
;; Server pushes events; client consumes them.
;; Cursor enables resumption after disconnection.

session Firehose
  ;; Connection phase: client optionally provides a cursor to resume from
  ! Int? .                              ;; cursor (none = start from current)
  +>
    | :accepted -> FirehoseStream
    | :cursor-too-old ->
        ? String .                      ;; info message: cursor outside backfill windo
        FirehoseStream                  ;; stream from earliest available
    | :error ->
        ? FirehoseError . end

session FirehoseStream
  rec
```

```
    ? SyncEvent .                        ;; server pushes an event
    +>
      | :continue -> FirehoseStream      ;; keep consuming
      | :disconnect -> end               ;; client disconnects

type FirehoseError
  := :consumer-too-slow                  ;; client can't keep up
   | :future-cursor                      ;; cursor is ahead of server
   | :internal-error String
```

7.3 6.3 Full Repository Sync Session

```
;; com.atproto.sync.getRepo - download a full repository as a CAR file.
;; Used for initial sync or resynchronization after broken commit chains.

session RepoExport
  ! DID                                  ;; which repo to export
  ! TID?                                 ;; since revision (none = full export)
  ? [Result Bytes RepoExportError]       ;; CAR file bytes
  end

type RepoExportError
  := :repo-not-found
   | :repo-deactivated
   | :repo-taken-down
   | :export-too-large
```

7.4 6.4 Relay Aggregation (Multi-Source Composition)

A Relay subscribes to multiple PDS firehoses and produces a single aggregated stream. This is protocol composition at the infrastructure level:

```
;; A Relay is a process that:
;; 1. Subscribes to N upstream PDS firehoses (as client)
;; 2. Produces one aggregated firehose (as server)
;; The Relay's SERVER-side session is the same Firehose type -
;; consumers can't distinguish a PDS firehose from a Relay firehose.

;; Relay's upstream subscription (one per PDS)
session RelayUpstream
  ! String .                             ;; PDS URL
```

```
  ! Int? .                              ;; cursor
  +>
    | :connected -> RelayUpstreamStream
    | :failed -> ? String . end

session RelayUpstreamStream
  rec
    ? SyncEvent .                       ;; events from this PDS
    +>
      | :continue -> RelayUpstreamStream
      | :pds-disconnect -> end          ;; upstream PDS disconnected
      | :reconnect ->                   ;; re-establish with new cursor
          ! Int . RelayUpstream

;; Relay's downstream service is simply: Firehose
;; (same type, same protocol - consumers see no difference)
```

7.5 6.5 Analysis: Protocol Reuse

The Relay demonstrates a key property of protocol composition: **the same session type serves different roles at different points in the architecture**. A `Firehose` session type is used by:

- PDS → Relay (upstream sync)
- Relay → App View (downstream aggregation)
- PDS → Direct consumer (direct subscription)

The session type is the same in all cases. The process behind it differs, but the protocol contract is identical. This is the session type analog of interface polymorphism — different implementations, same type.

The `RelayUpstreamStream` adds reconnection logic (`:reconnect` branches back to `RelayUpstream`) that the basic `FirehoseStream` doesn't need. This is composition through branching: the Relay's client-side protocol is a **superset** of the basic consumer protocol, with additional operational modes.

8 7. Account Lifecycle as Session Types

Account management — creation, migration, deactivation, deletion — is a state machine with well-defined transitions.

8.1 7.1 Account State Machine

```
;; Account states as a session type state machine.
;; Each state is a named session type; transitions are operations.

session AccountActive {admin :0 AdminCap}
  +>
    | :deactivate   -> AccountDeactivated
    | :delete       -> AccountDeleted
    | :suspend      -> AccountSuspended
    | :takedown     -> AccountTakenDown
    | :migrate      -> AccountMigration
    | :operate      -> RepoSession        ;; normal operations
    | :close-admin  -> end

session AccountDeactivated {admin :0 AdminCap}
  +>
    | :reactivate -> AccountActive
    | :delete     -> AccountDeleted
    | :close      -> end

session AccountSuspended {admin :0 AdminCap}
  +>
    | :unsuspend -> AccountActive
    | :delete    -> AccountDeleted
    | :close     -> end

session AccountTakenDown {admin :0 AdminCap}
  +>
    | :restore -> AccountActive          ;; takedowns are potentially reversible
    | :delete  -> AccountDeleted
    | :close   -> end

session AccountDeleted
  end                                    ;; terminal - no operations possible
```

8.2 7.2 Account Migration (Cross-PDS)

```
;; Account migration is a multi-phase protocol between old PDS, new PDS,
;; and the account holder. This is the most complex composition in ATP.
```

```
session AccountMigration {owner :0 AccountOwnerCap}
  ;; Phase 1: Create account on new PDS (may start deactivated)
  ! String .                              ;; new PDS URL
  ? [Result MigrationTicket MigrationError] .
  +>
    | :ticket-granted ->
        ;; Phase 2: Export repository from old PDS
        ! DID .
        ? [Result Bytes MigrationError] . ;; full CAR export
        +>
          | :export-ok ->
              ;; Phase 3: Import repository to new PDS
              ! Bytes .                   ;; CAR data
              ? [Result Unit MigrationError] .
              +>
                | :import-ok ->
                    ;; Phase 4: Update DID document (new PDS, new signing key)
                    ! DIDDocument .       ;; updated document
                    ? [Result Unit MigrationError] .
                    +>
                      | :did-updated ->
                          ;; Phase 5: Deactivate on old PDS, activate on new
                          ? MigrationComplete . end
                      | :did-failed -> MigrationRollback
                | :import-failed -> MigrationRollback
          | :export-failed -> end
    | :ticket-denied -> end

session MigrationRollback
  ? [Result Unit MigrationError]          ;; attempt to restore previous state
  end

type MigrationTicket
  :new-pds     String
  :signing-key SigningKey
  :expires     String                     ;; ISO 8601

type MigrationComplete
  :old-pds     String
```

```
  :new-pds     String
  :did         DID

type MigrationError
  := :pds-unreachable
   | :account-exists                    ;; already exists on target PDS
   | :export-failed
   | :import-validation-failed
   | :did-update-failed
   | :rollback-failed
```

8.3 7.3 Analysis: Capability Tracking Through Migration

Account migration is particularly interesting for capability analysis. The `{owner :0 AccountOwnerCap}` constraint ensures only the account owner can initiate migration. But notice how the protocol crosses PDS boundaries:

- Phase 1-2: old PDS (export) — requires read authority on old PDS
- Phase 3-4: new PDS (import) — requires write authority on new PDS
- Phase 4: DID directory — requires DID update authority

In a full Prologos realization, each phase would carry distinct capability requirements. The composed protocol's capability set would be the union: `{old-pds :0 ExportCap, new-pds :0 ImportCap, did :0 DIDUpdateCap}`.

The `MigrationRollback` session type handles the failure case — if DID update or import fails, the protocol transitions to rollback. This is error handling as a protocol phase, not as an exception.

9 8. Blob Management as Session Types

Blobs (media files) are uploaded separately from records and referenced by CID.

```
;; Blob upload and reference lifecycle

session BlobUpload {auth :0 AuthCap}
  ! String .                            ;; Content-Type MIME
  ! Bytes .                             ;; blob data
  ? [Result BlobRef BlobError]
```

```
  end

type BlobRef
  :cid       CID
  :mime-type String
  :size      Int

type BlobError
  := :too-large                          ;; exceeds size limit
   | :invalid-type                       ;; disallowed MIME type
   | :upload-failed
   | :storage-full

;; Blob retrieval
session BlobDownload
  ! DID . ! CID .                        ;; account + blob CID
  ? [Result Bytes BlobError]
  end
```

10 9. Lexicon Validation as a Type-Level Concern

Lexicons define the schema for records and XRPC endpoints. In ATP, Lexicon validation happens at record creation time. In Prologos, we can express this as a **dependent** session type — the record schema depends on the collection NSID.

```
;; Dependent record creation: the record type depends on the collection.
;; This uses dependent send (`:`) to bind the collection in the continuation.

session TypedRecordCreate {auth :0 AuthCap, write :0 RepoWriteCap}
  ?: collection Collection               ;; dependent receive: bind collection
  ! [RecordFor collection]               ;; record type depends on collection!
  ? [Result RepoRecord RepoError]
  end

;; RecordFor is a type family mapping collection NSIDs to record types:
;; RecordFor "app.bsky.feed.post"    = PostRecord
;; RecordFor "app.bsky.feed.like"    = LikeRecord
;; RecordFor "app.bsky.graph.follow" = FollowRecord
;; etc.
```

This is where Prologos's dependent session types (`?:` / `!:` operators) go beyond what ATP itself can express. ATP validates records against Lexicon schemas at runtime (JSON Schema-like validation). Prologos encodes the schema in the type system — invalid records are **type errors**, caught at compile time.

10.1 9.1 Lexicon Schema as Type (Sketch)

```
;; A Bluesky post record, typed
type PostRecord
  :text        String
  :created-at  String                    ;; ISO 8601 datetime
  :langs       [List String]?            ;; BCP 47 language tags
  :reply       ReplyRef?
  :embed       Embed?
  :labels      Labels?
  :tags        [List String]?
  :facets      [List Facet]?

type ReplyRef
  :root   StrongRef
  :parent StrongRef

type StrongRef
  :uri ATUri
  :cid CID

type Embed
  := :images [List ImageEmbed]
   | :external ExternalEmbed
   | :record StrongRef
   | :record-with-media StrongRef [List ImageEmbed]

;; A like record
type LikeRecord
  :subject    StrongRef
  :created-at String

;; A follow record
type FollowRecord
```

```
  :subject    DID
  :created-at String
```

11 10. The Full Client Session: Composition at Scale

Now we bring it all together. A full ATP client session composes identity resolution, authentication, repository operations, blob management, and sync subscription into a single session type.

11.1 10.1 Client PDS Full Session

```
;; The full client-to-PDS protocol, composed from all sub-protocols.
;; This is the capstone: every phase is a named session type,
;; and the full interaction is a type-checked state machine.

session ATPClientSession
  ;; Phase 1: Identity resolution
  ! Handle .
  ? [Result VerifiedIdentity ResolveError] .
  +>
    | :identity-ok ->
        ;; Phase 2: OAuth authentication
        OAuthFlow                        ;; transitions to AuthenticatedSession on succ
    | :identity-failed -> end

;; Recall: OAuthFlow on success transitions to AuthenticatedSession,
;; which offers:
;;   :xrpc-query    -> XRPCQuery . AuthenticatedSession
;;   :xrpc-procedure -> XRPCProcedure . AuthenticatedSession
;;   :refresh-token -> TokenRefresh
;;   :logout        -> end

;; A higher-level view that names the domain operations:

session BlueskyClient {auth :0 AuthCap}
  rec
    +>
      | :create-post   -> CreatePost . BlueskyClient
```

```
      | :like-post     -> LikePost . BlueskyClient
      | :follow-user   -> FollowUser . BlueskyClient
      | :get-timeline  -> GetTimeline . BlueskyClient
      | :get-profile   -> GetProfile . BlueskyClient
      | :upload-image  -> BlobUpload . BlueskyClient
      | :subscribe     -> Firehose
      | :logout        -> end

session CreatePost
  ! PostRecord
  ? [Result RepoRecord RepoError]
  end

session LikePost
  ! LikeRecord
  ? [Result RepoRecord RepoError]
  end

session FollowUser
  ! FollowRecord
  ? [Result RepoRecord RepoError]
  end

session GetTimeline
  ! Int . ! String? .                    ;; limit, cursor
  ? [Result [CursorResponse [List PostRecord]] RepoError]
  end

session GetProfile
  ! AtIdentifier .                       ;; DID or handle
  ? [Result ProfileView RepoError]
  end

type ProfileView
  :did           DID
  :handle        Handle
  :display-name  String?
  :description   String?
  :avatar        String?                 ;; blob URL
  :followers     Int
```

```
  :following      Int
  :posts         Int
```

11.2 10.2 The Protocol Graph

The full `ATPClientSession` forms a protocol graph:

```
ATPClientSession

  ! Handle . ? VerifiedIdentity



OAuthFlow  end

   AuthServerDiscovery
   PushedAuthRequest
   TokenExchange


AuthenticatedSession

   :xrpc-query  XRPCQuery
   :xrpc-procedure  XRPCProcedure
   :refresh-token  TokenRefresh
   :logout  end

BlueskyClient

   :create-post  CreatePost
   :like-post  LikePost
   :follow-user  FollowUser
   :get-timeline  GetTimeline
   :get-profile  GetProfile
   :upload-image  BlobUpload
   :subscribe  Firehose
   :logout  end
```

Each box is a named session type. Each arrow is a type-checked transition. The entire protocol — from handle resolution through OAuth through CRUD through firehose subscription — is a single verified state machine.

12 11. Federation as Multi-Party Protocol Composition

The AT Protocol's federation model involves multiple parties: Client PDS Relay App View. Each pair communicates via session-typed channels.

```
;; The federation topology as process composition.
;; Each arrow is a session-typed channel.

;; Client  PDS: ATPClientSession (or BlueskyClient)
;; PDS  Relay: Firehose (PDS is server, Relay is client)
;; Relay  App View: Firehose (Relay is server, App View is client)
;; Client  App View: XRPC queries (proxied through PDS)

defproc pds-node [config : PDSConfig] {sys : IOCap}
  ;; Accept client connections
  let client-ch := listen config.port ATPClientSession NetCap
  ;; Connect to relay for firehose publishing
  let relay-ch := connect config.relay-url Firehose NetCap

  rec
    +>
      | :client-request ->
          let ch := client-ch ?
          spawn handle-client ch AuthCap RepoWriteCap
          rec
      | :repo-event ->
          ;; Publish commit events to relay
          relay-ch ! [make-sync-event latest-commit]
          rec

defproc relay-node [config : RelayConfig] {sys : IOCap}
  ;; Subscribe to all known PDS firehoses
  let upstreams := [map (fn [pds-url]
    connect pds-url Firehose NetCap) config.pds-urls]

  ;; Serve aggregated firehose to downstream consumers
  let downstream-ch := listen config.port Firehose NetCap

  ;; Merge upstream events, publish downstream
```

```
rec
  let event := [select-any upstreams]    ;; whichever PDS sends next
  downstream-ch ! event
  rec
```

12.1 11.1 Analysis: Same Protocol, Different Roles

The `Firehose` session type appears three times in the federation:

1. **PDS publishes** a firehose (server-side of `Firehose`)
2. **Relay subscribes** to PDS firehoses (client-side of `dual Firehose`)
3. **Relay publishes** an aggregated firehose (server-side of `Firehose`)

The session type is identical in all cases. Duality handles the role reversal automatically: what the PDS sends (`? SyncEvent`), the Relay receives (`! SyncEvent` in the dual). What the Relay publishes downstream uses the same type, with App Views as the dual endpoints.

This is the power of protocols-as-types in a real federation: **the same type specification governs every firehose in the network**, regardless of which service implements it.

13 12. Key Findings

13.1 12.1 Every ATP Sub-Protocol Maps to a Session Type

AT Protocol Layer	Session Type(s)	Co
Identity	HandleResolve, DIDResolve, IdentityVerification	Sec
OAuth	AuthServerDiscovery, PushedAuthRequest, TokenExchange, TokenRefresh	Sec
XRPC	XRPCQuery, XRPCProcedure, PaginatedQuery	Ch
Repository	RepoSession, TypedRecordCreate	Ch
Sync/Firehose	Firehose, FirehoseStream, RepoExport	Ser
Account	AccountActive/Deactivated/Suspended/TakenDown/Deleted	Sta
Migration	AccountMigration, MigrationRollback	Mu
Blobs	BlobUpload, BlobDownload	Sin
Application	BlueskyClient, CreatePost, LikePost, etc.	Hu

Every layer of the AT Protocol can be expressed as one or more named session types. Complex protocols compose from simpler ones through named continuations.

13.2 12.2 Composition Patterns Observed

The AT Protocol exercise reveals several composition patterns:

1. **Sequential Phase Composition**: Identity resolution, OAuth flow, account migration. Each phase must complete before the next begins. Named continuations enforce ordering.

2. **Hub-and-Spoke**: `BlueskyClient`, `AuthenticatedSession`, `AccountActive`, `RepoSession`. A central session type branches to sub-protocols and returns. Mutual recursion between the hub and spokes creates a reusable operational loop.

3. **Server-Push Stream**: `FirehoseStream`. The server pushes events; the client's only choices are `:continue` or `:disconnect`. This is a specialized pattern for real-time data feeds.

4. **Cursor-Based Resumption**: `Firehose` accepts an optional cursor at connection time for resumption. `PaginatedQuery` threads cursors through recursive page fetches. Cursors are protocol state carried between sessions.

5. **State Machine**: Account lifecycle. Each state is a named session type; transitions are operations. `AccountDeleted` is terminal (`end`). This directly implements the ATP account status model as a type-checked state machine.

6. **Rollback Branch**: `AccountMigration` branches to `MigrationRollback` on failure. Error recovery is a named protocol phase, not an exception.

13.3 12.3 What Session Types Catch That ATP Cannot

The AT Protocol specification is informal prose supplemented by Lexicon schemas (JSON Schema-like). It describes protocol flows in natural language and enforces schema conformance at runtime. Session types would catch additional classes of errors:

Error Class	ATP Today	With Session Types
Out-of-order operations	Runtime error / undefined	Compile-time type error
Missing error handling	Silent failure	Type checker rejects
Skipped auth phase	401 at runtime	Can't construct the channel
Double-use of token	Undefined behavior	Linear type error
Broken commit chain	Runtime detection + re-sync	Type tracks revision
Migration phase skipped	Undefined state	Can't reach next phase
Firehose consumer lag	Connection dropped	Protocol-level backpressure

13.4 12.4 Dependent Types Add Schema Safety

The `TypedRecordCreate` session (§9) demonstrates something ATP cannot express today: the record type depends on the collection NSID. In ATP, Lexicon validation is a runtime JSON Schema check. In Prologos, the dependency is in the type:

```
?: collection Collection
! [RecordFor collection]                ;; compile-time: wrong record = type error
```

This eliminates an entire class of runtime errors — posting a `LikeRecord` to the `app.bsky.feed.post` collection is a **type error**, not a 400 response.

13.5 12.5 Capability Integration Is Natural

Every session type that requires authentication carries `{auth :0 AuthCap}`. Write operations carry `{write :0 RepoWriteCap}`. Admin operations carry `{admin :0 AdminCap}`. The capabilities compose:

- `BlueskyClient` requires `{auth :0 AuthCap}`
- `CreatePost` (reached through `BlueskyClient`) inherits the auth cap
- `AccountActive` requires `{admin :0 AdminCap}` (separate authority)
- `AccountMigration` requires `{owner :0 AccountOwnerCap}` (owner-only)

The capability requirements are visible in the type, auditable by the compiler, and erased at runtime. This is exactly the POLA (Principle of Least Authority) model from `CAPABILITY_SECURITY.md` applied to a real distributed protocol.

14 13. Implications for Prologos

14.1 13.1 Protocols-as-Types Is Practical

This exercise demonstrates that protocols-as-types composition is not merely theoretical — it scales to a real-world protocol with 10+ interacting sub-protocols, multiple transport mechanisms (HTTP, WebSocket), complex authentication flows, and federated multi-party interactions.

14.2 13.2 Session Type Libraries Enable Protocol Ecosystems

If Prologos provides a standard library of protocol building blocks (`Handshake`, `CursorPagination`, `TokenRefresh`, `ServerPushStream`), users can assemble application protocols by composition. The AT Protocol case study suggests a reusable set:

- `CursorPaginated` — cursor-based pagination loop
- `ServerPush` — server-push stream with client disconnect
- `OAuthBase` — PAR + PKCE + DPoP flow
- `StateMachine` — hub-and-spoke with named states
- `MultiPhaseWithRollback` — sequential phases with rollback branch

14.3 13.3 Federation Is Natural

The AT Protocol's federation model — same protocol, different roles, duality handling role reversal — maps perfectly to session type duality. Prologos's existing duality mechanism handles federation without any new infrastructure.

14.4 13.4 Dependent Sessions Enable Schema-Safe APIs

The `TypedRecordCreate` session demonstrates the value of dependent send/receive (`!:~`/?:~). Schema-typed APIs catch structural errors at compile time. This is a strong argument for prioritizing dependent send/receive implementation (Phase C in the IO Library roadmap).

14.5 13.5 The Intentionality Principle Holds

Throughout this exercise, the intentionality principle from `PROTOCOLS_AS_TYPES.org` proved its worth. Protocols that end with `end` are genuinely terminal (`AccountDeleted`, `BlobUpload`). Protocols that end with named continuations are genuinely transitional (`OAuthFlow → AuthenticatedSession`). The distinction was never ambiguous. End-splicing was never needed.

15 14. References

- AT Protocol Specification: `https://atproto.com/specs/atp`
- XRPC Specification: `https://atproto.com/specs/xrpc`
- Repository Specification: `https://atproto.com/specs/repository`
- Event Stream (Firehose): `https://atproto.com/specs/event-stream`
- Sync Specification: `https://atproto.com/specs/sync`
- OAuth for AT Protocol: `https://atproto.com/specs/oauth`
- DID Specification: `https://atproto.com/specs/did`
- Handle Specification: `https://atproto.com/specs/handle`
- Lexicon Schema: `https://atproto.com/specs/lexicon`
- Account Specification: `https://atproto.com/specs/account`
- Bluesky Documentation: `https://docs.bsky.app/docs/advanced-guides/atproto`

15.1 Prologos References

- Protocols as Types: `docs/tracking/principles/PROTOCOLS_AS_TYPES.org`
- Capability Security: `docs/tracking/principles/CAPABILITY_SECURITY.md`
- Session Type Design: `docs/tracking/2026-03-03_SESSION_TYPE_DESIGN.md`
- IO Library Design V2: `docs/tracking/2026-03-05_IO_LIBRARY_DESIGN_V2.md`